

Esame OS Internals (Cabodi)

Data: 04/09/2023

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Paginazione, Working Set e Page Fault)

Contesto: Si consideri il seguente frammento di programma:

```
#define N 32
data_t M[2*N][N];
int i, j, k;

for (k = 0; k <= 4; k++) {
    for (i = 0; i < 2*N; i++) {
        for (j = k; j < N; j += 4) {
            M[i][j].diag = i - j;
        }
    }
}
```

Il codice macchina generato dal programma viene eseguito su un sistema con gestione della memoria basata su paginazione a richiesta, pagine da 2KB, sostituzione delle pagine guidata da una politica Working set (versione esatta), con $\delta = 10$. Si sa che:

1. `sizeof(data_t) = 1024`
2. `data_t` ha un campo `diag` di tipo `int` (dimensione 32 bit)
3. il segmento di codice (istruzioni macchina) ha dimensioni inferiori a una pagina
4. `M` viene allocato a partire dall'indirizzo logico `0x5820C800`
5. la matrice `M` viene allocata seguendo la strategia "row major", cioè per righe (prima riga, seguita da seconda riga, ...).

Rispondere alle seguenti domande:

- A) Quante pagine (e frame) sono necessarie per memorizzare la matrice?
- A) Supponendo che le variabili `i`, `j` e `k` siano allocate nei registri (l'accesso ad esse non produce riferimenti di memoria), quanti riferimenti totali $NT = NW + NR$ (NR per la lettura e NW per la scrittura dei dati) produce il programma proposto (non considerare i fetch delle istruzioni)?
- A) Calcolare il numero di page fault generati dal programma proposto (motivare/giustificare la risposta).

- A) Completando correttamente il seguente programma, è possibile fornire un codice equivalente (in termini di comportamento) al precedente, che produce meno page fault. Completare il programma e calcolare quanti page fault vengono generati (motivare la risposta).

```
#define N 32
data_t M[2*N][N], *V;
int k;

V = &(M[0][0]);
for (k = 0; k < 2*N*N; k++) {
    V[k].diag = /* completare qui */;
}
```

Domanda 2 (File System Unix-like e Inode)

Contesto: Si consideri un file system Unix-like, basato su inode, con 15 puntatori/indici (12 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori/indici hanno dimensione 32 bit e i blocchi su disco hanno dimensioni 4KB. Il file system risiede su una partizione del disco, dove 2 TB sono riservati ai blocchi di dati. Lo spazio aggiuntivo riservato ai metadati (inclusi i blocchi di indice) può essere trascurato ai fini di questo esercizio.

- A) È noto che il file più piccolo nel File System ha dimensioni 8MB e il file più grande ha dimensioni 8GB. Si utilizzi N_2 per il numero di file con indice indiretto doppio e N_3 per il numero di file con indiretto triplo. Calcolare i valori massimi (i limiti superiori) per N_2 e N_3 .
- A) Dato un file binario di dimensioni 20490,5 KB, calcolare esattamente quanti blocchi di indice e di dato usa il file. Calcolare anche la frammentazione interna (per i blocchi di dato).
- A) Si consideri lo stesso file della domanda B. Si supponga di effettuare la chiamata `lseek(fd, offset, SEEK_END)` per posizionare l'offset del file (`SEEK_END` significa che l'offset viene riferito alla fine del file) per la successiva operazione di lettura/scrittura. Supponiamo che `fd` sia il descrittore del file associato al file (già aperto) e che `offset = -220`. Calcolare il numero di blocco logico (relativo ai blocchi logici del file, numerati a partire da 0) in corrispondenza del quale viene spostato l'offset.
-

Domanda 3 (Prestazioni e Allocazione File)

- A) Un file di testo di grandi dimensioni contiene N righe di lunghezza fissa (50 caratteri per ogni riga). Occorre ordinare le righe nel file usando un algoritmo di ordinamento di complessità lineartmica ($O(N \log N)$). A causa delle dimensioni, l'ordinamento viene implementato direttamente sul file (sfruttando memory mapping del file oppure utilizzando in modo opportuno le primitive di posizionamento, lettura e scrittura su file). A seconda del file system sottostante (strategia di allocazione dei file), quale complessità si prevede per l'ordinamento? (Scegliere tra $O(N)$, $O(N \log N)$, $O(N^2)$, $O(N^2 \log N)$, $O(N^3)$ per allocazione Contigua, Linked list, FAT, Inodes, e fornire una breve motivazione).

- A)** Un grande insieme di dati viene memorizzato su file utilizzando la strategia denominata "indice e file relativo" ("index and relative file"): il file indice contiene una sequenza di N record di dimensione fissa, ciascuno contenente una chiave di ricerca e un puntatore (un indirizzo logico) al record corrispondente nel file relativo. Il file di indice viene ordinato in base a chiavi di valore crescente. Il file relativo contiene N record di lunghezza variabile. I record NON sono ordinati. A seconda del file system sottostante (strategia di allocazione dei file), quale complessità si prevede per l'operazione di stampa di tutti i dati, seguendo l'ordine delle chiavi? (Scegliere tra $O(\log N)$, $O(N)$, $O(N \log N)$, $O(N^2)$ per allocazione Contigua, Linked list, FAT, Inodes, e fornire una breve motivazione).

2 SECTION 2: OS161

Domanda 4 (Sincronizzazione OS161 - Producer/Consumer)

Contesto: Si considerino tre thread del kernel OS161 che implementano un'attività di trasferimento dati basata su un modello producer/consumer (2 producer, 1 consumer). I thread condividono un buffer di dati, implementato come struttura C di tipo `struct prodConsBuf`, definito come segue:

```
struct prodConsBuf {
    data_t data;
    int dataReady;
    struct lock *pc_lk;
    struct cv *pc_cv;
};
```

Tutte le operazioni (scrittura/lettura), necessitano di mutua esclusione. Ogni volta che nel buffer non sono presenti dati (quindi il buffer è vuoto), il flag `dataReady` è 0. Dopo che un produttore scrive nuovi dati nel buffer (campo `data`), pone `dataReady` al valore 1 (quindi buffer pieno) e segnala il consumer (il lavoro viene eseguito chiamando la funzione `producerWrite`). Il consumer esegue in modo iterativo la funzione `consumerRead`.

Rispondere alle seguenti domande:

- B) La struttura condivisa può essere posizionata nello stack di thread o deve essere una variabile globale o altro?
- B) Poiché i campi `pc_lk` e `pc_cv` sono puntatori, dove dovrebbero essere chiamate le funzioni `lock_create()` e `cv_create()`? Nel thread producer, nel consumer thread? Altrove?
- B) Le funzioni di produttore e consumatore sono parzialmente realizzate e riportate di seguito. Completare la loro implementazione dove indicato (<1>, <2> e <3>). Si noti che occorre evitare "busy waiting", quindi un produttore segnala al consumatore quando i dati sono pronti. Poiché un produttore potrebbe dover attendere che il consumatore legga ("consumi") i dati precedenti, prima che il produttore sia autorizzato a scrivere un nuovo dato, anche il consumatore deve fare una segnalazione.

```
/* error checking/handling instructions are omitted for simplicity */
void producerWrite (struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire (pc->pc_lk);

    /* properly handle here the fact that the buffer could be full
       (filled by the other producer and not yet read by the consumer) */
    <1>

    /* copy data: done here by simple assignment
       a proper function could be used instead */
    pc->data = *dp;
    pc->dataReady = 1; /* set flag */

    /* signal the consumer */
    <2>

    /* just release the lock and return */
```

```

        lock_release (pc->pc_lk);
    }

void consumerRead (struct prodConsBuf *pc, data_t *dp) {
    /* lock for mutual exclusion */
    lock_acquire (pc->pc_lk);

    while (!pc->dataReady) {
        cv_wait(pc->pc_cv, pc->pc_lk);
    }

    /* copy data: done here by simple assignment
       a proper function could be used instead */
    *dp = pc->data;
    pc->dataReady = 0; /* reset set flag */

    /* signal a producer waiting, if any */
    <3>

    /* release the lock and return */
    lock_release (pc->pc_lk);
}

```

Domanda 5 (System Calls: open, close, write in OS161)

Si consideri una possibile implementazione delle chiamate di sistema `open()` e `close()` in OS161.

- B) La "per-process file table" è allocata in memoria user o in memoria kernel?
- B) Si supponga che due processi utente, in esecuzione contemporaneamente, chiamino `write(fd, v, N)`, dove `fd` è 5 per entrambi, ci si aspetta che (per ogni opzione rispondere SI/NO e motivare):
1. I processi scrivano sullo stesso file.
 2. Non scrivano mai sullo stesso file.
 3. Generalmente scrivano su file diversi ma a volte potrebbero scrivere sullo stesso.
 4. Per scrivere sullo stesso file, i due processi dovrebbero utilizzare valori diversi per `fd`.
- B) Data la seguente implementazione di `sys_write`:

```

int sys_write (int fd, userptr_t buf_ptr, size_t size) {
    int i;
    char *p = (char *)buf_ptr;

    if (fd != STDOUT_FILENO && fd != STDERR_FILENO) {
        return file_write(fd, buf_ptr, size);
    }

    for (i = 0; i < (int)size; i++) {
        putchar(p[i]);
    }
}

```

```
    }  
    return (int)size;  
}
```

Supponendo che standard input, output ed error siano mappati alla console (quindi non è possibile alcun loro reindirizzamento a file), si dica se è possibile sostituire la gestione di `stdout/stderr` mediante il ciclo `for` con uno o più dei seguenti frammenti di codice. (Per ogni alternativa rispondere SI/NO e fornire una motivazione).

```
1) for (i=0; i<(int)size; i++) {  
    kprintf("%c", p[i]);  
}  
2) kprintf("%s", p);  
3) kprintf("%s", &p);  
4) file_write(fd, p, size);
```